

PRIMENA GRASP METAHEURISTIKE ZA REŠAVANJE TWO-STAGE FIXED CHARGE TRANSPORTATION PROBLEMA SA AUTOMATSKIM PRIKUPLJANJEM METRIKA

Ime i Prezime
Broj indeksa: 0000/2024

Mart 2025.

Sažetak

U ovom radu razmatra se Two-Stage Fixed Charge Transportation Problem (TSFCTP), koji predstavlja proširenje klasičnog transportnog problema uvođenjem fiksnih troškova korišćenja veza u obe faze transporta. Problem se sastoji od tri nivoa: izvori, distributivni centri i kupci, pri čemu se roba najpre transportuje od izvora do distributivnih centara, a zatim od distributivnih centara do kupaca. Za svaku vezu u obe faze definisani su varijabilni troškovi (proporcionalni količini) i fiksni troškovi (koji se plaćaju jednokratno ako se veza koristi). Cilj je minimizirati ukupne troškove uz zadovoljenje ponude i potražnje.

U radu je implementirana GRASP (Greedy Randomized Adaptive Search Procedure) metaheuristika sa poboljšanjima, uključujući adaptivnu konstrukcijsku fazu koja uzima u obzir fiksne troškove i agresivno lokalno pretraživanje sa tri različite strategije. Pored toga, implementiran je sistem za automatsko prikupljanje svih potrebnih metrika u skladu sa zahtevima izrade izveštaja: pamćenje najboljih rešenja kroz više izvršavanja, vremena do pronalaska najboljeg rešenja, ukupna vremena izvršavanja, kao i izračunavanje srednjih vrednosti i standardnih devijacija. Algoritam automatski učitava optimalna rešenja iz literature za poređenje i generiše izveštaj u tekstualni fajl.

Ključne reči: TSFCTP, GRASP, fiksni troškovi, lokalno pretraživanje, metaheuristika, prikupljanje metrika, analiza performansi

Sadržaj

1	Uvod	3
2	Opis problema i matematička formulacija	3
2.1	Definicija problema	3
2.2	Matematička formulacija	4
2.3	Oblasti primene	4
2.4	Postojeći načini rešavanja	5
3	GRASP metaheuristika	5
3.1	Osnovni koncept	5
3.2	Pseudokod GRASP algoritma	5
3.3	Adaptacija GRASP-a na TSFCTP problem	6
3.3.1	Predstavljanje rešenja	6
3.3.2	Funkcija cilja	6
4	Detaljan opis implementacije	6
4.1	Klasa za podatke - TSFCTPData	6
4.2	Funkcije za učitavanje instanci i optimalnih rezultata	7
4.2.1	Funkcija read_instance	7
4.2.2	Funkcija get_results_filename	8
4.2.3	Funkcija load_optimal_for_instance	8
4.2.4	Funkcija get_report_filename	9
4.3	Klasa za rešenje - TSFCTPSolution	9
4.4	Konstruktivska faza - ImprovedGRASPConstruction	10
4.4.1	Metoda construct	10
4.4.2	Metoda _evaluate_candidates	11
4.5	Lokalno pretraživanje - AggressiveLocalSearch	12
4.5.1	Glavna petlja - metoda improve	12
4.5.2	Strategija 1: Konsolidacija tokova (_consolidate_flows)	12
4.5.3	Strategija 2: Prebacivanje DC-ova (_switch_dc)	13
4.5.4	Strategija 3: Gašenje neprofitabilnih veza (_remove_unprofitable_links)	14
4.6	Klasa za prikupljanje metrika - GRASPWithOptimalMetrics	15
4.6.1	Inicijalizacija	15
4.6.2	Metoda single_run	15
4.6.3	Metoda run_multiple	17
4.6.4	Metoda _compute_statistics	17
4.6.5	Metoda print_report	18
4.7	Glavni program	19
5	Rezultati testiranja, analiza rezultata i poređenja	20
6	Zaključak	20

1 Uvod

Transportni problemi predstavljaju jednu od najznačajnijih klasa problema u operacionim istraživanjima. Klasični transportni problem bavi se optimalnim raspoređivanjem robe od izvora do odredišta uz minimizaciju troškova transporta, pri čemu su troškovi proporcionalni količini transportovane robe. Međutim, u realnim situacijama često se susrećemo sa fiksnim troškovima koji ne zavise od količine, već se plaćaju jedno-kratno ako se određena veza koristi.

Two-Stage Fixed Charge Transportation Problem (TSFCTP) predstavlja prirodno proširenje koje uključuje fiksne troškove u obe faze transporta. Problem je NP-težak, što znači da za velike instance ne postoji algoritam koji može garantovano pronaći optimalno rešenje u polinomijalnom vremenu. Stoga se za rešavanje ovakvih problema koriste metaheuristički pristupi.

Cilj ovog rada je implementacija GRASP metaheuristike za rešavanje TSFCTP problema, sa posebnim naglaskom na sistem za automatsko prikupljanje metrika koje su potrebne za izradu izveštaja. Implementirani sistem prati sve relevantne parametre tokom izvršavanja i generiše strukturiran izveštaj u tekstualni fajl.

2 Opis problema i matematička formulacija

2.1 Definicija problema

TSFCTP se može opisati na sledeći način: Imamo tri nivoa u mreži snabdevanja:

- I izvora (sources) sa ograničenom ponudom s_i , $i = 1, \dots, I$
- J distributivnih centara (DC-ova) koji služe kao međustanice
- K kupaca (customers) sa potražnjom d_k , $k = 1, \dots, K$

Roba se transportuje u dve faze:

1. Prva faza: od izvora i do distributivnog centra j
2. Druga faza: od distributivnog centra j do kupca k

Za svaku vezu u prvoj fazi (i, j) definisani su:

- Varijabilni trošak b_{ij} po jedinici robe
- Fiksni trošak f_{ij} koji se plaća ako se veza koristi (bez obzira na količinu)

Za svaku vezu u drugoj fazi (j, k) definisani su:

- Varijabilni trošak c_{jk} po jedinici robe
- Fiksni trošak g_{jk} koji se plaća ako se veza koristi

2.2 Matematička formulacija

Uvodimo sledeće promenljive:

- x_{ij} - količina robe transportovana od izvora i do DC-a j
- y_{jk} - količina robe transportovana od DC-a j do kupca k
- $z_{ij} \in \{0, 1\}$ - indikator korišćenja veze (i, j)
- $w_{jk} \in \{0, 1\}$ - indikator korišćenja veze (j, k)

Minimizovati:

$$\min \sum_{i=1}^I \sum_{j=1}^J (b_{ij}x_{ij} + f_{ij}z_{ij}) + \sum_{j=1}^J \sum_{k=1}^K (c_{jk}y_{jk} + g_{jk}w_{jk}) \quad (1)$$

uz ograničenja:

$$\sum_{j=1}^J x_{ij} \leq s_i, \quad \forall i \in \{1, \dots, I\} \quad (2)$$

$$\sum_{j=1}^J y_{jk} = d_k, \quad \forall k \in \{1, \dots, K\} \quad (3)$$

$$\sum_{i=1}^I x_{ij} = \sum_{k=1}^K y_{jk}, \quad \forall j \in \{1, \dots, J\} \quad (4)$$

$$x_{ij} \leq M \cdot z_{ij}, \quad \forall i, j \quad (5)$$

$$y_{jk} \leq M \cdot w_{jk}, \quad \forall j, k \quad (6)$$

$$x_{ij}, y_{jk} \geq 0, \quad z_{ij}, w_{jk} \in \{0, 1\} \quad (7)$$

Ograničenje (2) osigurava da izvori ne isporuče više od svoje ponude. Ograničenje (3) zahteva da potražnja svakog kupca bude tačno zadovoljena. Ograničenje (4) obezbeđuje konzistentnost tokova kroz distributivne centre (ulaz jednako izlaz). Ograničenja (5) i (6) povezuju kontinuirane i binarne promenljive (M je dovoljno veliki broj).

2.3 Oblasti primene

TSFCTP ima široku primenu u realnim problemima:

- **Logistika i lanac snabdevanja:** Planiranje distribucije robe od proizvođača preko regionalnih skladišta do maloprodajnih objekata
- **Telekomunikacije:** Postavljanje mrežnih čvorova i uspostavljanje veza uz fiksne troškove opreme
- **Energetika:** Distribucija električne energije od elektrana preko transformatorskih stanica do potrošača
- **Proizvodnja:** Planiranje proizvodnje i distribucije uz fiksne troškove pokretanja proizvodnih linija

2.4 Postojeći načini rešavanja

Za rešavanje TSFCTP problema razvijeni su različiti pristupi:

- **Precizni algoritmi:** Branch-and-bound, branch-and-cut, dinamičko programiranje - primenljivi samo za male instance
- **Matheuristike:** Kombinacija matematičkog programiranja i heuristika
- **Metaheuristike:** Genetski algoritmi, simulirano kaljenje, algoritmi rojeva, GRASP

3 GRASP metaheuristika

3.1 Osnovni koncept

GRASP (Greedy Randomized Adaptive Search Procedure) je višeiterativna metaheuristika koju su razvili Feo i Resende 1995. godine. Svaka iteracija GRASP algoritma sastoji se iz dve faze:

1. **Konstrukcijska faza:** Kreira se izvodljivo rešenje korišćenjem pohlepno-slučajnog algoritma
2. **Faza lokalnog pretraživanja:** Istražuje se okolina dobijenog rešenja u potrazi za boljim

Ključna karakteristika GRASP-a je upotreba RCL (Restricted Candidate List) koja omogućava balans između pohlepnosti i slučajnosti, čime se postiže raznolikost u konstruisanim rešenjima.

3.2 Pseudokod GRASP algoritma

Algorithm 1 Osnovni GRASP algoritam

```
1: procedure GRASP(MaxIterations, Seed)
2:   BestSolution  $\leftarrow \emptyset$ 
3:   BestCost  $\leftarrow \infty$ 
4:   for  $k = 1$  to MaxIterations do
5:     Solution  $\leftarrow$  GreedyRandomizedConstruction(Seed)
6:     Solution  $\leftarrow$  LocalSearch(Solution)
7:     if  $\text{Cost}(\textit{Solution}) < \textit{BestCost}$  then
8:       BestSolution  $\leftarrow$  Solution
9:       BestCost  $\leftarrow$   $\text{Cost}(\textit{Solution})$ 
10:    end if
11:  end for
12:  return BestSolution
13: end procedure
```

3.3 Adaptacija GRASP-a na TSFCTP problem

Za potrebe rešavanja TSFCTP problema, GRASP algoritam je prilagođen na sledeći način:

3.3.1 Predstavljanje rešenja

Rešenje je predstavljeno kroz dve matrice celih brojeva:

- x_{ij} - tok od izvora i do DC-a j
- y_{jk} - tok od DC-a j do kupca k

Veze se smatraju aktivnim ako je odgovarajući tok veći od nule, što automatski implicira plaćanje fiksnog troška.

3.3.2 Funkcija cilja

Ukupni trošak se računa kao:

$$\text{Cost} = \underbrace{\sum_{i,j} b_{ij}x_{ij} + \sum_{j,k} c_{jk}y_{jk}}_{\text{varijabilni troškovi}} + \underbrace{\sum_{i,j:x_{ij}>0} f_{ij} + \sum_{j,k:y_{jk}>0} g_{jk}}_{\text{fiksni troškovi}} \quad (8)$$

4 Detaljan opis implementacije

4.1 Klasa za podatke - TSFCTPData

Klasa TSFCTPData služi za čuvanje svih ulaznih podataka instance problema. Implementirana je korišćenjem `dataclass` dekoratora koji automatski generiše osnovne metode.

```
1 @dataclass
2 class TSFCTPData:
3     I: int # broj izvora
4     J: int # broj distributivnih centara
5     K: int # broj kupaca
6     s: np.ndarray # ponuda izvora
7     d: np.ndarray # potraznja kupaca
8     b: np.ndarray # varijabilni troskovi i->j
9     f: np.ndarray # fiksni troskovi i->j
10    c: np.ndarray # varijabilni troskovi j->k
11    g: np.ndarray # fiksni troskovi j->k
12
13    def validate(self):
14        """Provera konzistentnosti podataka"""
15        assert len(self.s) == self.I
16        assert len(self.d) == self.K
17        assert self.b.shape == (self.I, self.J)
18        assert self.f.shape == (self.I, self.J)
19        assert self.c.shape == (self.J, self.K)
```

```

20     assert self.g.shape == (self.J, self.K)
21     assert np.sum(self.s) >= np.sum(self.d)
22     assert np.all(self.s == self.s.astype(int))
23     assert np.all(self.d == self.d.astype(int))

```

Listing 1: Klasa TSFCTPData

Metoda `validate()` proverava konzistentnost učitanih podataka i osigurava da su sve dimenzije matrica odgovarajuće.

4.2 Funkcije za učitavanje instanci i optimalnih rezultata

4.2.1 Funkcija `read_instance`

Funkcija `read_instance` učitava instancu problema iz tekstualnog fajla. Format fajla je sledeći:

- Prva linija: broj izvora I, broj DC-ova J, broj kupaca K
- Druga linija: ponuda izvora (I brojeva)
- Treća linija: potražnja kupaca (K brojeva)
- Zatim I redova sa J brojeva (matrica b)
- Zatim I redova sa J brojeva (matrica f)
- Zatim J redova sa K brojeva (matrica c)
- Zatim J redova sa K brojeva (matrica g)

```

1 def read_instance(filename):
2     with open(filename, 'r') as f:
3         nums = [int(x) for line in f for x in line.split()]
4
5         it = iter(nums)
6         I, J, K = next(it), next(it), next(it)
7         s = np.array([next(it) for _ in range(I)], dtype=int)
8         d = np.array([next(it) for _ in range(K)], dtype=int)
9         b = np.array([[next(it) for _ in range(J)] for _ in range
10                        (I)], dtype=int)
11         f = np.array([[next(it) for _ in range(J)] for _ in range
12                        (I)], dtype=int)
13         c = np.array([[next(it) for _ in range(K)] for _ in range
14                        (J)], dtype=int)
15         g = np.array([[next(it) for _ in range(K)] for _ in range
16                        (J)], dtype=int)
17
18         data = TSFCTPData(I, J, K, s, d, b, f, c, g)
19         data.validate()
20     return data

```

Listing 2: Funkcija za učitavanje instance

4.2.2 Funkcija `get_results_filename`

Ova funkcija na osnovu putanje do instance određuje koji fajl sa rezultatima treba učitati. Prepoznaje prefikse `small_`, `medium_` i `large_`.

```
1 def get_results_filename(instance_path):
2     filename = os.path.basename(instance_path)
3
4     if filename.startswith('small_'):
5         return "results_small.txt"
6     elif filename.startswith('medium_'):
7         return "results_medium.txt"
8     elif filename.startswith('large_'):
9         return "results_large.txt"
10    else:
11        return None
```

Listing 3: Određivanje fajla sa rezultatima

4.2.3 Funkcija `load_optimal_for_instance`

Funkcija učitava optimalno rešenje i vreme za konkretnu instancu tako što:

1. Određuje odgovarajući fajl sa rezultatima pomoću `get_results_filename`
2. Otvara fajl i traži red koji počinje putanjom do instance
3. Parsira treću kolonu (optimalna vrednost) i četvrtu kolonu (optimalno vreme)

```
1 def load_optimal_for_instance(instance_path):
2     results_file = get_results_filename(instance_path)
3
4     try:
5         with open(results_file, 'r') as f:
6             for line in f:
7                 parts = line.strip().split()
8                 if len(parts) < 4:
9                     continue
10
11                 if parts[0] == instance_path:
12                     optimal_value = float(parts[2])
13                     optimal_time = float(parts[3])
14                     return optimal_value, optimal_time
15     except FileNotFoundError:
16         print(f"Fajl {results_file} nije pronadjen.")
17
18     return None, None
```

Listing 4: Učitavanje optimalnog rešenja

4.2.4 Funkcija get_report_filename

Generiše ime izveštajnog fajla na osnovu instance, npr. za `small_3.txt` generiše `GRASP_results/GRASP_report_small_3.txt`.

```
1 def get_report_filename(instance_path):
2     filename = os.path.basename(instance_path)
3     name_without_ext = os.path.splitext(filename)[0]
4     return f"GRASP_results/GRASP_report_{name_without_ext}.txt"
```

Listing 5: Generisanje imena izveštajnog fajla

4.3 Klasa za rešenje - TSFCTPSolution

Klasa `TSFCTPSolution` predstavlja jedno rešenje problema. Sadrži matrice tokova x i y , kao i metode za izračunavanje troškova i proveru validnosti.

```
1 class TSFCTPSolution:
2     def __init__(self, data: TSFCTPData):
3         self.data = data
4         self.x = np.zeros((data.I, data.J), dtype=int)
5         self.y = np.zeros((data.J, data.K), dtype=int)
6         self.total_cost = 0
7         self.var_cost = 0
8         self.fixed_cost = 0
9
10    def calculate_cost(self):
11        """Izracunava ukupni trosak ukljucujuci fiksne"""
12        self.var_cost = np.sum(self.x * self.data.b) + np.sum(
13            (self.y * self.data.c)
14
15        self.fixed_cost = 0
16        for i in range(self.data.I):
17            for j in range(self.data.J):
18                if self.x[i, j] > 0:
19                    self.fixed_cost += self.data.f[i, j]
20
21        for j in range(self.data.J):
22            for k in range(self.data.K):
23                if self.y[j, k] > 0:
24                    self.fixed_cost += self.data.g[j, k]
25
26        self.total_cost = self.var_cost + self.fixed_cost
27        return self.total_cost
```

Listing 6: Klasa `TSFCTPSolution` - osnovne metode

Metoda `is_valid()` proverava da li rešenje zadovoljava sva ograničenja:

- Da ukupna količina poslata iz svakog izvora ne prelazi ponudu
- Da je potražnja svakog kupca tačno zadovoljena

- Da je za svaki DC ulaz jednak izlazu
- Da nema negativnih tokova

4.4 Konstrukcijska faza - ImprovedGRASPConstruction

Klasa ImprovedGRASPConstruction implementira poboljšanu konstrukcijsku fazu koja uzima u obzir fiksne troškove.

4.4.1 Metoda construct

Glavna metoda koja konstruiše početno rešenje. Redosled operacija:

1. Inicijalizacija preostalih količina i skupa aktivnih veza
2. Sortiranje kupaca po potražnji (veći prvi) - za bolju konsolidaciju
3. Za svakog kupca, dok god ima nezadovoljene potražnje:
 - Evaluacija svih mogućih ruta
 - Formiranje RCL liste
 - Nasumičan odabir rute
 - Određivanje količine (veća za nove veze)
 - Ažuriranje rešenja i preostalih količina

```

1 def construct(self) -> TSFCTPSolution:
2     solution = TSFCTPSolution(self.data)
3     remaining_supply = self.data.s.copy()
4     remaining_demand = self.data.d.copy()
5     active_first = set()
6     active_second = set()
7
8     customers = list(range(self.data.K))
9     customers.sort(key=lambda k: self.data.d[k], reverse=True)
10
11     for k in customers:
12         need = remaining_demand[k]
13         while need > 0:
14             candidates = self._evaluate_candidates(k,
15                 remaining_supply,
16                 active_first, active_second)
17             # RCL i odabir...
18             i, j = random.choice(rcl)
19
20             max_possible = min(remaining_supply[i], need)
21
22             if (i, j) in active_first:
23                 amount = max_possible

```

```

23         else:
24             min_amount = max(1, max_possible // 5)
25             amount = random.randint(min_amount,
26                                     max_possible)
27
28             solution.x[i, j] += amount
29             solution.y[j, k] += amount
30             active_first.add((i, j))
31             active_second.add((j, k))
32
33             remaining_supply[i] -= amount
34             need -= amount

```

Listing 7: Konstrukcijska faza - glavna petlja

4.4.2 Metoda `_evaluate_candidates`

Ova metoda je ključna za uvažavanje fiksnih troškova u konstrukciji. Za svaku potencijalnu rutu (i,j) za kupca k računa:

$$\text{score} = b_{ij} + c_{jk} + \text{penalizacija} \quad (9)$$

gde je penalizacija:

- Ako veza (i,j) nije aktivna: $f_{ij}/\text{expected_flow}$
- Ako veza (j,k) nije aktivna: $g_{jk}/\text{expected_flow}$

Očekivani tok je $\min(\text{remaining_supply}[i], \text{data.d}[k])$ - procena koliko bi robe moglo ići ovom rutom.

```

1  def _evaluate_candidates(self, k, remaining_supply,
2      active_first, active_second):
3
4      candidates = []
5
6      for i in range(self.data.I):
7          if remaining_supply[i] <= 0:
8              continue
9
10         for j in range(self.data.J):
11             var_cost = self.data.b[i, j] + self.data.c[j, k]
12             fixed_penalty = 0.0
13             expected_flow = min(remaining_supply[i], self.
14                                 data.d[k])
15
16             if expected_flow > 0:
17                 if (i, j) not in active_first:
18                     fixed_penalty += self.data.f[i, j] /
19                                     expected_flow
20
21                 if (j, k) not in active_second:

```

```

18         fixed_penalty += self.data.g[j, k] /
           expected_flow
19
20     score = var_cost + fixed_penalty
21     score += random.uniform(0, 0.1) # mali random š
           um
22     candidates.append((score, (i, j)))
23
24     candidates.sort(key=lambda x: x[0])
25     return candidates

```

Listing 8: Evaluacija kandidata sa fiksnim troškovima

4.5 Lokalno pretraživanje - AggressiveLocalSearch

Klasa `AggressiveLocalSearch` implementira tri strategije lokalnog pretraživanja koje se primenjuju redom dok god ima poboljšanja.

4.5.1 Glavna petlja - metoda `improve`

```

1 def improve(self, solution):
2     current = solution.copy()
3     improved = True
4     iteration = 0
5     max_iterations = 100
6
7     while improved and iteration < max_iterations:
8         improved = False
9         iteration += 1
10
11         if self._consolidate_flows(current):
12             improved = True
13             continue
14
15         if self._switch_dc(current):
16             improved = True
17             continue
18
19         if self._remove_unprofitable_links(current):
20             improved = True
21             continue
22
23     return current

```

Listing 9: Glavna petlja lokalnog pretraživanja

4.5.2 Strategija 1: Konsolidacija tokova (`_consolidate_flows`)

Ova strategija pokušava da spoji tokove iz više izvora u isti DC na najjeftiniji izvor. Za svaki DC j , pronalazi sve izvore koji šalju robu u njega. Ako ih ima više od

jednog, sortira ih po varijabilnom trošku i pokušava sav tok prebaciti na najjeftiniji izvor. Ušteda dolazi od gašenja fiksnih troškova ostalih veza.

```

1 def _consolidate_flows(self, solution):
2     for j in range(self.data.J):
3         sources = [(i, solution.x[i, j]) for i in range(self.
4             data.I)
5                 if solution.x[i, j] > 0]
6
7         if len(sources) <= 1:
8             continue
9
10        sources.sort(key=lambda x: self.data.b[x[0], j])
11        best_i = sources[0][0]
12
13        current_fixed = sum(self.data.f[i, j] for i, _ in
14            sources)
15        new_fixed = self.data.f[best_i, j]
16        fixed_saving = current_fixed - new_fixed
17
18        if fixed_saving > 0:
19            # Zapamti staro stanje
20            old_values = [(i, solution.x[i, j]) for i, _ in
21                sources[1:]]
22
23            # Prebaci sve na best_i
24            for i, flow in sources[1:]:
25                solution.x[best_i, j] += flow
26                solution.x[i, j] = 0
27
28            if solution.is_valid():
29                return True
30            else:
31                # Vрати назад
32                for i, flow in old_values:
33                    solution.x[best_i, j] -= flow
34                    solution.x[i, j] = flow
35
36        return False

```

Listing 10: Konsolidacija tokova

4.5.3 Strategija 2: Prebacivanje DC-ova (_switch_dc)

Ova strategija pokušava da prebaci tokove s jednog DC-a na drugi. Za svaki par DC-ova j_1 i j_2 , za svakog kupca k koji prima od j_1 , računa promenu troška:

$$\Delta = \text{flow} \cdot (c_{j_2,k} - c_{j_1,k}) + \Delta_{\text{fiksni}} \quad (10)$$

gde je Δ_{fiksni} ušteda od gašenja veze j_1 - k (ako se gasi) minus trošak aktiviranja veze j_2 - k (ako se aktivira).

```

1 def _switch_dc(self, solution):
2     for j1 in range(self.data.J):
3         for j2 in range(self.data.J):
4             if j1 == j2:
5                 continue
6
7             for k in range(self.data.K):
8                 flow = solution.y[j1, k]
9                 if flow == 0:
10                    continue
11
12                 var_diff = flow * (self.data.c[j2, k] - self.
13                     data.c[j1, k])
14
15                 fixed_diff = 0
16                 if solution.y[j1, k] > 0 and np.sum(solution.
17                     y[j1, :]) == flow:
18                     fixed_diff -= self.data.g[j1, k]
19
20                 if solution.y[j2, k] == 0:
21                     fixed_diff += self.data.g[j2, k]
22
23                 total_diff = var_diff + fixed_diff
24
25                 if total_diff < 0: # ušteda
26                     # Pronađi izvor i prebaci tok
27                     for i in range(self.data.I):
28                         if solution.x[i, j1] >= flow:
29                             # Prebaci...
30                             return True
31
32     return False

```

Listing 11: Prebacivanje DC-ova

4.5.4 Strategija 3: Gašenje neprofitabilnih veza (`_remove_unprofitable_links`)

Ova strategija identifikuje veze sa malim tokovima (manje od 3 jedinice) i visokim fiksnim troškovima (većim od 100) i pokušava da ih ugasi prebacivanjem tokova na druge aktivne veze.

```

1 def _remove_unprofitable_links(self, solution):
2     for i in range(self.data.I):
3         for j in range(self.data.J):
4             if solution.x[i, j] == 0:
5                 continue
6
7             if solution.x[i, j] < 3 and self.data.f[i, j] >
8                 100:
9                 for j2 in range(self.data.J):
10                    if j2 == j:

```

```

10         continue
11
12         if solution.x[i, j2] > 0:
13             flow = solution.x[i, j]
14             # Prebaci tok na j2...
15             return True
16     return False

```

Listing 12: Gašenje neprofitabilnih veza

4.6 Klasa za prikupljanje metrika - GRASPWithOptimalMetrics

Ovo je glavna klasa koja integriše sve prethodne komponente i prikuplja potrebne metrike za izveštaj.

4.6.1 Inicijalizacija

```

1 class GRASPWithOptimalMetrics:
2     def __init__(self, data, instance_path, alpha=0.2,
3                 max_iterations_per_run=200, platform_info="
4                     AMD_Ryzen_5,_16GB_RAM"):
5
6         self.data = data
7         self.instance_path = instance_path
8         self.alpha = alpha
9         self.max_iterations_per_run = max_iterations_per_run
10        self.platform_info = platform_info
11
12        # Učitaj optimalno rešenje
13        self.optimal_solution, self.optimal_time =
14            load_optimal_for_instance(instance_path)
15
16        self.results = []
17        self.report_filename = get_report_filename(
18            instance_path)

```

Listing 13: Inicijalizacija GRASPWithOptimalMetrics

4.6.2 Metoda single_run

Ova metoda izvršava jedan run GRASP algoritma i prikuplja:

- `best_cost` - najbolje rešenje u ovom run-u (sol_i)
- `time_to_best` - vreme kada je prvi put dostignuto najbolje rešenje (t_i)
- `total_time` - ukupno vreme trajanja run-a ($ttot_i$)
- `iterations` - broj iteracija ($iter_i$)

```

1 def single_run(self, run_id, seed):
2     random.seed(seed)
3     np.random.seed(seed)
4
5     start_time = time.time()
6
7     best_solution = None
8     best_cost = float('inf')
9     best_time_found = 0
10    iteration_count = 0
11
12    alphas = [0.1, 0.2, 0.3, 0.4]
13
14    for iteration in range(self.max_iterations_per_run):
15        alpha = alphas[iteration % len(alphas)]
16
17        constructor = ImprovedGRASPConstruction(self.data,
18            alpha)
19        solution = constructor.construct()
20
21        local_search = AggressiveLocalSearch(self.data)
22        solution = local_search.improve(solution)
23
24        if not solution.is_valid():
25            continue
26
27        cost = solution.calculate_cost()
28        iteration_count += 1
29
30        if cost < best_cost:
31            best_cost = cost
32            best_solution = solution.copy()
33            best_time_found = time.time() - start_time
34
35    total_time = time.time() - start_time
36
37    return {
38        'run_id': run_id,
39        'seed': seed,
40        'best_solution': best_solution,
41        'best_cost': best_cost,
42        'time_to_best': best_time_found,
43        'total_time': total_time,
44        'iterations': iteration_count
45    }

```

Listing 14: Jedno izvršavanje GRASP algoritma

4.6.3 Metoda run_multiple

Pokreće algoritam više puta (zadati broj run-ova) i prikuplja rezultate.

```
1 def run_multiple(self, num_runs=10, verbose=True):
2     self.results = []
3
4     for run in range(num_runs):
5         seed = 42 + run * 100
6         result = self.single_run(run, seed)
7         self.results.append(result)
8
9     return self._compute_statistics()
```

Listing 15: Višestruko izvršavanje

4.6.4 Metoda _compute_statistics

Ova metoda računa sve potrebne statistike prema zahtevu:

- **Bestsol** - najbolje rešenje (ako postoji optimalno, koristi se ono)
- $\text{gap}_i = 100 \cdot \frac{|\text{sol}_i - \text{Bestsol}|}{|\text{Bestsol}|}$ za svaki run
- $t = \frac{1}{k} \sum_{i=1}^k t_i$ - srednje početno vreme
- $\text{ttot} = \frac{1}{k} \sum_{i=1}^k \text{ttot}_i$ - srednje ukupno vreme
- $\text{agap} = \frac{1}{k} \sum_{i=1}^k \text{gap}_i$ - srednje odstupanje
- $\sigma = \sqrt{\frac{1}{k} \sum_{i=1}^k (\text{gap}_i - \text{agap})^2}$ - standardna devijacija

```
1 def _compute_statistics(self):
2     all_costs = [r['best_cost'] for r in self.results]
3     best_ma = min(all_costs)
4
5     # Odredi Bestsol (optimalno ako postoji)
6     if self.optimal_solution is not None:
7         bestsol = self.optimal_solution
8     else:
9         bestsol = best_ma
10
11     # Izračunaj gap za svaki run
12     gaps = []
13     for r in self.results:
14         gap = 100 * abs(r['best_cost'] - bestsol) / abs(
15             bestsol)
16         r['gap'] = gap
17         gaps.append(gap)
18
19     stats = {
```

```

19         'num_runs': len(self.results),
20         'bestsol': bestsol,
21         'optimal_solution': self.optimal_solution,
22         'optimal_time': self.optimal_time,
23         'best_ma_solution': best_ma,
24         'mean_time_to_best': np.mean([r['time_to_best'] for r
25                                     in self.results]),
26         'mean_total_time': np.mean([r['total_time'] for r in
27                                     self.results]),
28         'mean_gap': np.mean(gaps),
29         'std_gap': np.std(gaps),
30         'mean_iterations': np.mean([r['iterations'] for r in
31                                     self.results]),
32         'all_results': self.results
33     }
34
35     return stats

```

Listing 16: Izračunavanje statistika

4.6.5 Metoda print_report

Generiše izveštaj i čuva ga u fajl. Izveštaj sadrži:

- Informacije o instanci (dimenzije, putanja)
- Optimalno rešenje iz literature (ako postoji)
- Informacije o platformi
- Tabelu sa rezultatima svih run-ova
- Srednje vrednosti i standardnu devijaciju

```

1 def print_report(self, stats):
2     report_lines = []
3     report_lines.append("=" * 80)
4     report_lines.append("IZVEŠTAJ_O_IZVRŠAVANJU_GRASP_
5                           METAHEURISTIKE")
6     report_lines.append("=" * 80)
7
8     # Informacije o instanci
9     report_lines.append("\n---INSTANCA---")
10    report_lines.append(f"Putanja: {self.instance_path}")
11    report_lines.append(f"Dimenzije: I={self.data.I}, J={self
12                        .data.J}, K={self.data.K}")
13
14    if self.optimal_solution:
15        report_lines.append(f"\n---OPTIMALNO REŠENJE---")
16        report_lines.append(f"Optimalno rešenje: {self.
17                            optimal_solution:.2f}")

```

```

15         report_lines.append(f"Vreme_za_optimalno:{self.
            optimal_time:.2f}s")
16
17     # Tabela rezultata
18     report_lines.append(f"\n---_REZULTATI_({stats['num_runs
        ']}_izvršavanja)_---")
19     report_lines.append(f"\n{'Run':<6}_{'Best_Cost':<12}_{'
        Gap(%)':<10}_
20                             f"{'t(s)':<10}_{'ttot(s)':<10}_{'
                                Iter':<8}")
21     report_lines.append("-" * 70)
22
23     for i, r in enumerate(stats['all_results']):
24         report_lines.append(f"{i+1:<6}_{r['best_cost']:<12.0f
            }_{r['gap']:<10.2f}_
25             f"{r['time_to_best']:<10.2f}_{r['total_time
                ']:<10.2f}_{r['iterations']:<8}")
26
27     # Srednje vrednosti
28     report_lines.append(f"\n---_SREDNJE_VREDNOSTI_---")
29     report_lines.append(f"srednje_početno_vreme_(t)_{stats['mean_time_to_best']:.2f}s")
30     report_lines.append(f"srednje_ukupno_vreme_(ttot)_{stats['mean_total_time']:.2f}s")
31     report_lines.append(f"srednje_odstupanje_(agap)_{stats['mean_gap']:.2f}%")
32     report_lines.append(f"standardna_devijacija_{stats['std_gap']:.2f}%")
33
34     # Čuvanje u fajl
35     with open(self.report_filename, 'w', encoding='utf-8') as
        f:
36         for line in report_lines:
37             f.write(line + '\n')
38
39     print(f"\nIzveštaj_je_sačuvan_u:{self.report_filename}")

```

Listing 17: Generisanje izveštaja

4.7 Glavni program

Glavni program obrađuje argumente komandne linije, učitava instancu i pokreće GRASP algoritam.

```

1 if __name__ == "__main__":
2     if len(sys.argv) < 2:
3         print("Korišćenje: python grasp_report.py <
            datoteka_instance> [broj_izvršavanja]")
4         sys.exit(1)
5
6     filename = sys.argv[1]

```

```

7   num_runs = int(sys.argv[2]) if len(sys.argv) > 2 else 10
8
9   data = read_instance(filename)
10
11  grasp = GRASPWithOptimalMetrics(
12      data,
13      instance_path=filename,
14      alpha=0.2,
15      max_iterations_per_run=200,
16      platform_info="AMD Ryzen 5, 16GB RAM"
17  )
18
19  stats = grasp.run_multiple(num_runs=num_runs, verbose=
20      True)
    grasp.print_report(stats)

```

Listing 18: Glavni program

5 Rezultati testiranja, analiza rezultata i poređenja

6 Zaključak

U ovom radu implementirana je GRASP metaheuristika za rešavanje Two-Stage Fixed Charge Transportation problema. Implementacija obuhvata nekoliko ključnih komponenti:

1. **Klasu za podatke** koja omogućava jednostavno rukovanje ulaznim podacima i njihovu validaciju.
2. **Funkcije za učitavanje** koje omogućavaju automatsko prepoznavanje tipa instance (small/medium/large) i učitavanje odgovarajućih optimalnih rešenja iz literature.
3. **Klasu za rešenje** koja čuva tokove, izračunava troškove i proverava validnost rešenja.
4. **Poboljšanu konstrukcijsku fazu** koja kroz RCL mehanizam i penalizaciju fiksnih troškova omogućava balans između pohlepnosti i slučajnosti, istovremeno vodeći računa o fiksnim troškovima kroz amortizaciju.
5. **Agresivno lokalno pretraživanje** sa tri strategije:
 - Konsolidacija tokova - spajanje tokova na najjeftinije izvore
 - Prebacivanje DC-ova - premeštanje celih tokova na jeftinije distributivne centre
 - Gašenje neprofitabilnih veza - uklanjanje veza sa malim tokovima i visokim fiksnim troškovima

6. **Sistem za prikupljanje metrika** koji u potpunosti zadovoljava zahteve za izradu izveštaja:

- Pamćenje najboljih rešenja kroz više izvršavanja (sol_i)
- Merenje vremena do pronalaska najboljeg rešenja (t_i)
- Merenje ukupnog vremena izvršavanja ($ttot_i$)
- Brojanje iteracija ($iter_i$)
- Izračunavanje odstupanja u odnosu na optimalna rešenja (gap_i)
- Računanje srednjih vrednosti (t , $ttot$, $agap$) i standardne devijacije (σ)

7. **Automatsko generisanje izveštaja** u tekstualni fajl sa svim relevantnim informacijama.

Implementirani sistem omogućava pouzdano i ponovljivo izvršavanje eksperimenata, sa automatskim prikupljanjem svih metrika potrebnih za analizu performansi. Korišćenje dataclasses i tipiziranih struktura podataka olakšava održavanje i proširivanje koda.

Poseban značaj ima način na koji konstrukcijska faza tretira fiksne troškove - umesto da ih ignoriše, ona ih amortizuje kroz očekivani tok, što dovodi do formiranja rešenja sa manjim brojem aktivnih veza i boljom konsolidacijom tokova.

Lokalno pretraživanje je dizajnirano tako da prvo primenjuje strategije koje donose najveće uštede (konsolidacija), a zatim one finije (prebacivanje DC-ova i gašenje neprofitabilnih veza), čime se obezbeđuje efikasno pretraživanje prostora rešenja.

Literatura

- [1] H. I. Calvete, C. Galé, J. A. Iranzo, P. Toth, *A matheuristic for the two-stage fixed-charge transportation problem*.